Project Overview & System Architecture

1. Project Title & Mission

Smart Pill Box Reminder & Medication Safety Monitor with Embedded SD Logging & On-OLED Dosage History

The Smart Pill Box is an embedded medical-grade adherence device constructed on the **STM32F103C8T6 (ARM Cortex-M3)** microcontroller. Designed to bridge healthcare adherence and embedded safety, the system provides:

1. **Scheduled Medication Reminders**: Dual-alarm daily schedules (Pill 1 and Pill 2) configured with exact minutes and hours.

2. **Multi-Sensory Audio-Visual Alerting**: 0.96-inch monochrome I2C OLED display, dedicated pill indicator LEDs, and an acoustic piezo frequency alert.

3. **Active Error Prevention ("Wrong Pill" Safety Interlock)**: If a patient presses the wrong confirmation button during an alarm window, the device immediately switches to a high-frequency 150 ms acoustic alarm and displays an inverted video warning screen (`"WRONG PILL!"`), preventing double-dosing and incorrect medication intake.

4. **On-OLED 3-Day Dosage History Viewer (1 Page per Day)**: Allows the user or physician to review medication compliance for Today, Yesterday (-1d), and Day Before (-2d) directly on the screen without needing a computer.

5. **Persistent Storage & Audit Trail via MicroSD (SPI1)**: Saves alarm schedules to **Sector 1000** so settings survive power outages, and logs every confirmation and warning event to **Sector 1001+** with timestamps.

6. **Battery-Backed Real-Time Clock**: DS1307 RTC with CR2032 coin cell maintaining wall-clock time, with automatic fallback to MCU SysTick software clock interpolation if the RTC is unpowered.

7. **Complete On-Device 3-Button HMI**: 100% offline configuration (Pill 1 time, Pill 2 time, RTC clock adjustment, and log review).

---

2. Hardware Architecture & Block Diagram

```
graph TD
    subgraph Controller [STM32F103C8T6 ARM Cortex-M3 - 72 MHz]
        I2C1[I2C1 Peripheral: PB6 SCL, PB7 SDA - 100 kHz]
        SPI1[SPI1 Peripheral: PA4 CS, PA5 SCK, PA6 MISO, PA7 MOSI - 9 MHz]
        GPIO_IN[Push Buttons: PB12, PB14, PB15 with Internal Pull-Ups]
        GPIO_OUT[Indicators: PB1 LED1, PB10 LED2, PB0 Buzzer]
        SYS_TICK[SysTick 1ms Hardware Downcounter]
    end

    subgraph I2C_Bus [Shared I2C1 Bus - 3.3V Logic with Pull-Ups]
        OLED[SSD1306 128x64 OLED Display - Address: 0x3C / 0x78]
        RTC[DS1307 Real-Time Clock - Address: 0x68 / 0xD0]
    end

    subgraph SPI_Bus [SPI1 Flash Bus]
        SD[MicroSD Card Socket: Sector 1000 Config & Sector 1001+ Logs]
    end

    subgraph User_HMI [Human-Machine Interface]
        BTN1[Button 1 PB12: Dose 1 OK / Next Day / Menu Inc]
        BTN2[Button 2 PB14: Dose 2 OK / Select / Field Next]
        BTN3[Button 3 PB15: Setup Menu / Exit / Back]
    end

    subgraph Indicators [Visual & Acoustic Feedback]
        LED1[Pill 1 Green LED - PB1]
        LED2[Pill 2 Yellow LED - PB10]
        BUZZ[Piezo Buzzer - PB0]
    end

    subgraph Power_Subsystem [Power Regulation]
        V5V[5V Rail: DS1307 VCC, MicroSD VCC via LDO]
        V3V3[3.3V Rail: STM32 VDD, OLED VDD, SPI Logic]
        VBAT[CR2032 3.0V Coin Cell: DS1307 VBAT Backup]
    end

    I2C1 <==> I2C_Bus
    SPI1 <==> SPI_Bus
    GPIO_IN <== User_HMI
    GPIO_OUT ==> Indicators
    Power_Subsystem -.-> Controller
    Power_Subsystem -.-> I2C_Bus
    Power_Subsystem -.-> SPI_Bus
```

---

3. Comprehensive Wiring & Pinout Table

Peripheral	Signal	Pin	Electrical Mode	Voltage Level	Hardware Function
I2C1	**SCL**	**PB6**	AF Open-Drain (**GPIO_MODE_AF_OD**)	5V-Tolerant (FT)	Shared I2C clock line (100 kHz)
I2C1	**SDA**	**PB7**	AF Open-Drain (**GPIO_MODE_AF_OD**)	5V-Tolerant (FT)	Shared I2C data line
SPI1	**CS**	**PA4**	GPIO Push-Pull (**GPIO_MODE_OUTPUT_PP**)	3.3V Logic	SD Card Chip Select (Active LOW)
SPI1	**SCK**	**PA5**	AF Push-Pull (**GPIO_MODE_AF_PP**)	3.3V Logic	SPI1 Bus Clock (281 kHz init / 9 MHz transfer)

SPI1	**MISO**	**PA6**	Input Pull-Up (**GPIO_MODE_INPUT**)	3.3V Max (Non-FT)	Serial Data from SD card to MCU
SPI1	**MOSI**	**PA7**	AF Push-Pull (**GPIO_MODE_AF_PP**)	3.3V Logic	Serial Data from MCU to SD card
Button 1	**BTN1**	**PB12**	GPIO Input Pull-Up (**GPIO_PULLUP**)	5V-Tolerant (FT)	Pill 1 Confirm / Menu Increment (+) / Next Day
Button 2	**BTN2**	**PB14**	GPIO Input Pull-Up (**GPIO_PULLUP**)	5V-Tolerant (FT)	Pill 2 Confirm / Menu Select / Next Field
Button 3	**BTN3**	**PB15**	GPIO Input Pull-Up (**GPIO_PULLUP**)	5V-Tolerant (FT)	Setup Menu Open / Discard / Back
LED 1	**P1_LED**	**PB1**	GPIO Push-Pull (**GPIO_MODE_OUTPUT_PP**)	3.3V (20mA max)	Pill 1 Active Reminder Visual Output
LED 2	**P2_LED**	**PB10**	GPIO Push-Pull (**GPIO_MODE_OUTPUT_PP**)	3.3V (20mA max)	Pill 2 Active Reminder Visual Output
Buzzer	**BUZZ**	**PB0**	GPIO Push-Pull (**GPIO_MODE_OUTPUT_PP**)	3.3V Drive	Piezo Buzzer Acoustic Frequency Output
ST-Link	**SWDIO**	**PA13**	Serial Wire Data In/Out	3.3V Logic	Hardware In-Circuit Debugging & Flashing
ST-Link	**SWCLK**	**PA14**	Serial Wire Clock	3.3V Logic	Hardware Debugging Clock

---

4. Software State Machine Architecture

The firmware is structured as a deterministic, non-blocking cooperative super-loop based on a Finite State Machine (FSM):

```
stateDiagram-v2
    [*] --> BOOT_DIAGNOSTICS: Power-On Reset
    BOOT_DIAGNOSTICS --> NORMAL_IDLE: I2C Scan & SD Init (2.5s)

    state NORMAL_IDLE {
        [*] --> ClockMonitoring
        ClockMonitoring --> AlarmCheck: Every 500ms RTC Read
    }

    NORMAL_IDLE --> REMINDER_DOSE_1: Time == Dose 1 Alarm (e.g. 07:30)
    NORMAL_IDLE --> REMINDER_DOSE_2: Time == Dose 2 Alarm (e.g. 19:00)
    NORMAL_IDLE --> SETUP_MENU: Button 3 Pressed

    state REMINDER_DOSE_1 {
        [*] --> Alerting_P1: Normal 500ms Pulse
        Alerting_P1 --> Wrong_Alert_P1: Button 2 Pressed (Wrong!)
        Wrong_Alert_P1 --> Wrong_Alert_P1: 150ms Fast Pulse + Inverted Screen
    }

    state REMINDER_DOSE_2 {
        [*] --> Alerting_P2: Normal 500ms Pulse
        Alerting_P2 --> Wrong_Alert_P2: Button 1 Pressed (Wrong!)
        Wrong_Alert_P2 --> Wrong_Alert_P2: 150ms Fast Pulse + Inverted Screen
    }

    REMINDER_DOSE_1 --> DOSE_CONFIRMED: Button 1 Pressed (Correct)
    REMINDER_DOSE_2 --> DOSE_CONFIRMED: Button 2 Pressed (Correct)

    DOSE_CONFIRMED --> NORMAL_IDLE: 2.0s Success Screen + Log Recorded

    state SETUP_MENU {
        [*] --> MenuCycle: B1=Cycle (1:P1, 2:P2, 3:Clock, 4:Logs)
        MenuCycle --> MenuSelect: B2=Select Active Option
    }

    SETUP_MENU --> NORMAL_IDLE: Button 3 Pressed (Exit)
    SETUP_MENU --> SETUP_EDIT_HOUR: Option 1, 2, or 3 Selected
    SETUP_EDIT_HOUR --> SETUP_EDIT_MINUTE: Button 2 Pressed (B1 = +Hour)
    SETUP_EDIT_MINUTE --> SETUP_CONFIRM: Button 2 Pressed (B1 = +Min)
    SETUP_CONFIRM --> SETUP_FEEDBACK: Button 2 Confirmed (B1 = Toggle Save/Cancel)
    SETUP_FEEDBACK --> NORMAL_IDLE: 1.5s Display Delay (Sector 1000 Updated)

    SETUP_MENU --> VIEW_LOGS: Option 4 Selected
    state VIEW_LOGS {
        [*] --> Day0_Today: Page 1 (Today: P1 & P2 Status)
        Day0_Today --> Day1_Yesterday: Button 1 Pressed
        Day1_Yesterday --> Day2_DayBefore: Button 1 Pressed
        Day2_DayBefore --> Day0_Today: Button 1 Pressed
    }
    VIEW_LOGS --> SETUP_MENU: Button 3 Pressed (Back)
```

---

5. Critical Engineering Decisions: CSE vs. ECE Perspective

5.1 Why Open-Drain for I2C instead of Push-Pull?

In standard push-pull mode, a pin actively drives HIGH (connecting to V_{DD}) or LOW (connecting to GND). If two devices on a shared bus drive opposite logic levels simultaneously, a direct short-circuit occurs (V_{DD} to GND through low-impedance MOSFETs), destroying the silicon. Open-drain outputs only pull LOW (via an NMOS switch) or float in high-impedance (Hi-Z). Bus pull-up resistors passively pull the lines HIGH when all devices release the bus, creating a **wired-AND** configuration that permits multi-drop communication and clock stretching.

5.2 Why Hardware SPI1 for MicroSD instead of Bit-Banging?

Hardware SPI1 uses the STM32's dedicated master synchronous serial hardware, driven directly by $PCLK2$ (72 MHz). By configuring the prescaler to `/8`, the MCU achieves a transmission baud rate of **9 Mbit/s** (1.125 MB/s), completing a 512-byte sector read or write in less than 0.6 ms, ensuring zero perceptible latency in user interaction.

5.3 Deterministic Real-Time Behavior Without an RTOS

For small medical devices requiring fail-safe reliability, an RTOS adds task context-switch overhead, stack memory fragmentation, and priority inversion risks. Our system uses a **single super-loop cooperative architecture** driven by `HAL_GetTick()`. State transitions execute in under $50\,\mu\text{s}$, and time slicing guarantees that display updates, alarm checks, and debounce routines never block one another.